Seq2seq and Attention*

Viktoriia Zinkovich

In this homework we shall compose encoder-decoder neural networks and apply them to the **task of machine translation** (Figure 1). Namely, you will build a classic seq2seq system with attention for Russian–English translation.

Contents

1 Building vocabularies (2 points) **2**

2 Basic model (7 points) **3**
- 2.1 Encoder-decoder (3 points) . . . 3
- 2.2 Training loss (2 points) . . . 3
- 2.3 Training basic model (2 points) . . . 3

3 Your Attention Required (11 points) **3**
- 3.1 Attention layer (3 points) . . . 3
- 3.2 Seq2seq model with attention (3 points) . . . 4
- 3.3 Training attentive model (5 points) . . . 5
- 3.4 Visualizing model attention . . . 5

Encoder-decoder architectures are about converting anything to anything, including:

1. **Machine translation** and spoken dialogue systems
2. **Image captioning** and **image2latex** (convolutional encoder, recurrent decoder)
3. Generating **images by captions** (recurrent encoder, convolutional decoder)
4. Convert **words to transcripts** – Grapheme2phoneme

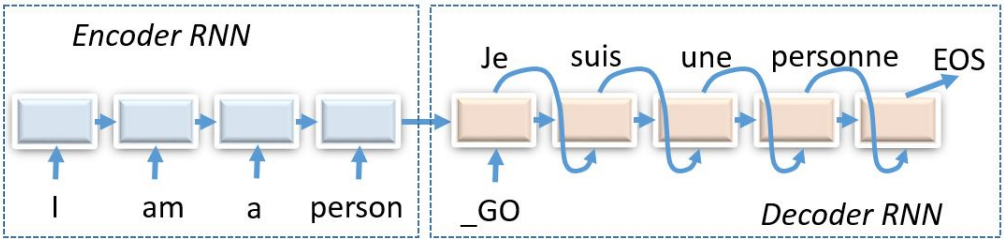

Figure 1: Schematic view of the RNN model

We are going to try our encoder-decoder models on Russian to English machine translation problem. More specifically, we'll translate **hotel and hostel descriptions**. This task shows the scale of machine translation while not requiring you to train your model for weeks if you don't use GPU.

*We gratefully acknowledge the authors of the "NLP Course for You" for providing the original code and datasets [1].

Environment Setup

To run the homework, install the required dependencies. The command below will install the necessary Python packages and download the translation dataset.

```
$ bash setup.sh
```

Tokenization

Before we get to the architecture, there's *some preprocessing to be done*. This time we've done preprocessing for you – the data will be tokenized with `WordPunctTokenizer` in `setup.sh`.

However, our data lines contain unique **rare words.** If we operate on a word level, we will have to deal with large vocabulary size. If instead we use character-level models, it would take lots of iterations to process a sequence. This time we are going to pick something in between.

One popular approach is called **Byte Pair Encoding** aka BPE (you can check files `train.bpe` after set up). The algorithm starts with a character-level tokenization and then iteratively merges most frequent pairs for N iterations. This results in frequent words being merged into a single token and rare words split into syllables or even characters.

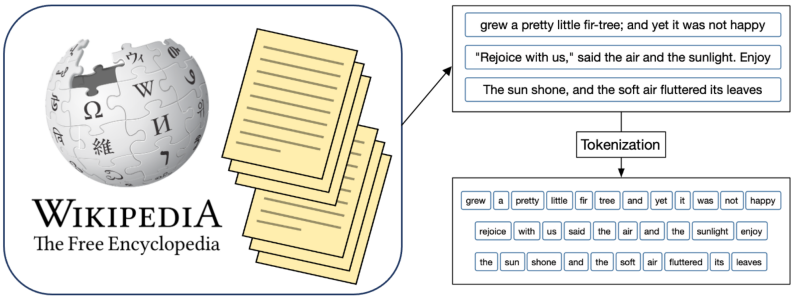

Figure 2: Schematic pipeline of tokenization process

1 Building vocabularies (2 points)

We now need to build vocabularies that map strings to token ids and vice versa. We're going to need these fellas when we feed training data into model or convert output matrices into words.

Implement the class `Vocab` in `vocab.py` file that:

- builds a vocabulary from a list of tokenized strings;
- contains special tokens `<BOS>`, `<EOS>`, `<UNK>`;
- provides `to_matrix(lines)` to convert a list of strings into a matrix of token ids (including BOS/EOS tokens);
- provides `to_lines(matrix)` to convert matrices of token ids back into tokenized strings;
- provides `compute_mask(matrix)` that returns a mask over output tokens and includes the first `EOS` but excludes all tokens after it.

You can check your implementation using the unit test:

```
$ ./run.py unittest vocab
```

2 Basic model (7 points)

2.1 Encoder-decoder (3 points)

The code in `seq2seq.py` contains a template for a simple encoder-decoder model: single GRU encoder/decoder, no attention or anything. Your task is to implement the `decode_step` method of `BasicModel`. It should return the new decoder state and the output logits.

You can check your implementation using the unit test:

```
$ ./run.py unittest basic_decode
```

2.2 Training loss (2 points)

Implement the `compute_loss` function in `training.py`. The training objective is:

$$L = \frac{1}{|D|} \sum_{X,Y \in D} \sum_{y_t \in Y} -\log p(y_t \mid y_1, \ldots, y_{t-1}, X, \theta)$$

where $|D|$ is the **total length of all sequences**, including BOS and first EOS, but excluding PAD.

You can check your implementation using the unit test:

```
$ ./run.py unittest loss
```

2.3 Training basic model (2 points)

Write the training loop for the `BasicModel` model in `training.py`. You should:

- create the `BasicModel` with appropriate parameters;
- train until convergence or achievement of indicated metrics;
- save the final checkpoint to `outputs/basic_model_checkpoint.pt`;
- report final BLEU and check that translation examples look valid.

The basic model should achieve at least **BLEU $>$ 10** on the dev set. Metric computation is done via `compute_bleu` function.

Important: The unit test will load your saved checkpoint and evaluate the BLEU score on the dev set. Make sure to save the checkpoint after training completes.

To train the model, you use the training script: `python training.py basic`.

You can check your implementation using the unit test:

```
$ ./run.py unittest basic_training
```

3 Your Attention Required (11 points)

In this section you will improve over the basic model by implementing a simple attention mechanism. This consists of two parts: building the **attention layer** and using it for an **attentive seq2seq model**.

3.1 Attention layer (3 points)

Implement the `AttentionLayer` class in `seq2seq.py`. The layer should compute additive attention. Given encoder sequence $h_0^e, h_1^e, h_2^e, \ldots, h_T^e$ and a single decoder state h^d:

Compute logits with a 2-layer neural network:

$$a_t = \text{linear}_{\text{out}}\big(\tanh(\text{linear}_e(h_t^e) + \text{linear}_d(h_d))\big)$$

Get probabilities from logits:

$$p_t = \frac{e^{a_t}}{\sum_\tau e^{a_\tau}}$$

Add up encoder states with probabilities to get attention response:

$$attn = \sum_t p_t \cdot h_t^e$$

You can check your implementation using the unit test:

```
$ ./run.py unittest attention_layer
```

3.2 Seq2seq model with attention (3 points)

You can now use the attention layer to build a network. The simplest way to implement attention is to use it in decoder phase:

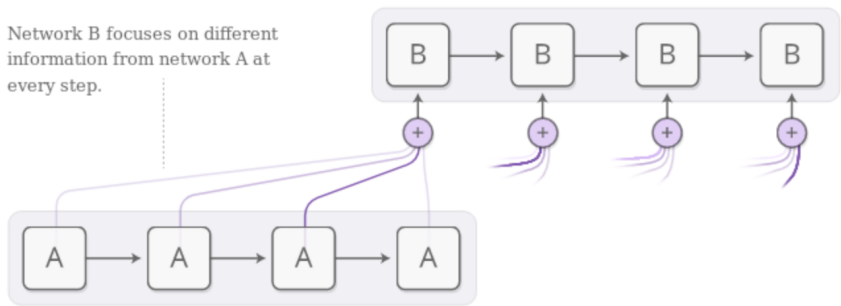

Figure 3: Schematic attention idea

On every step, use **previous** decoder state to obtain attention response. Then feed concatenation of this response to the inputs of next attention layer.

Your task now is to implement the `AttentiveModel` class in `seq2seq.py`. The key implementation detail here is **model state**. Put simply, you can add any tensor into the list of `encode` outputs. You will then have access to them at each `decode` step. This may include:

- Last RNN hidden states (as in basic model);
- The whole sequence of encoder outputs (to attend to) and mask;
- Attention probabilities (to visualize)

Tip

There are, of course, alternative ways to wire attention into your network and different kinds of attention. Take a look at this, this and this for ideas. And for image captioning/im2latex there's visual attention.

You can check your implementation using the unit test:

```
$ ./run.py unittest attentive_model
```

3.3 Training attentive model (5 points)

Write the training loop for the `AttentiveModel` model in `training.py`. You should:

- create the `AttentiveModel` with appropriate parameters;
- train until convergence or achievement of indicated metrics;
- save the final checkpoint to `outputs/attentive_model_checkpoint.pt`;
- report final BLEU and check that translation examples look valid.

The attentive model should achieve at least **BLEU** $>$ **20** on the dev set.

Important: The unit test will load your saved checkpoint and evaluate the BLEU score on the dev set. Make sure to save the checkpoint after training completes.

To train the model, you use the training script: `python training.py attentive`.

You can check your implementation using the unit test:

```
$ ./run.py unittest attentive_training
```

3.4 Visualizing model attention

After training the attentive translation model, you can check its sanity by visualizing its attention weights.

We provided you with a function that draws attention maps using `Bokeh`. Your task is to extract attention probabilities from your trained attentive model and visualize them. Specifically, you need to:

1. Extract attention probabilities from the model states returned by `translate_lines`. The attention probabilities should have shape `[batch_size, translation_length, input_length]`.
2. Use the provided `draw_attention` function to visualize the attention weights for first translation example from the dataset *(of course you can explore more, but you should send only one).*
3. Save the attention map (as screenshot or using Bokeh's export functionality) and submit it alongside this notebook as `attention_map.png`.

Note: You are not locked into using Bokeh. If you prefer a different visualization method (e.g., `matplotlib`, `seaborn`), feel free to use that instead.

Note: If the attention maps are not interpretable (look like random noise), try starting the encoder from zeros (instead of `dec_start`), forcing the model to use attention.

References

[1] Elena Voita. NLP Course For You, Sep 2020.